UGANDA CHRISTIAN UNIVERSITY
A Center of Excellence in the Heart of Africa
DEPARTMENT OF COMPUTING & TECHNOLOGY
Faculty of Engineering, Design & Technology

CSC2107 GROUP PROJECT

ALGORITHM EXPLORER

Design, Implementation and Testing

An Interactive Java Application for Exploring Algorithms and Data Structures

Programme: Bachelor of Science in Information Technology

Course Unit: Design and Analysis of Algorithms

Course Code: CSC2107

Institution: Uganda Christian University

GROUP MEMBERS

No.	Name	Registration Number
1	Okiror Erasmus	S25B23/088
2	Kyobutungi Tereza	S25B23/111
3	Nyamwenge Bernadette	S23B23/049
4	Aruho Bob	S24B23/115
5	Atuhaire Sheenah	S25B23/072

Executive Summary

Algorithm Explorer is a Java desktop application with a JavaFX graphical interface. It demonstrates sorting, recursion and manually implemented data structures using user-supplied inputs. The application presents results, execution steps, selected operation counts, elapsed time and theoretical complexity explanations in one workspace.

The Sorting Lab implements bubble, selection, insertion and merge sort. The Recursion Lab provides factorial, naive Fibonacci and recursive binary search. The remaining laboratories demonstrate an array-based stack, a circular array queue and a singly linked list built from custom nodes. The algorithm implementations do not call built-in sorting routines or use ready-made stack, queue or list implementations as their underlying structures.

The interface separates input controls, results, visual states and explanatory notes. Sorting and recursion use recorded steps for playback; structure operations show the resulting state, textual steps and operation history. A persistent sidebar retains each laboratory's state while the application remains open. The startup window was adjusted to use the available screen area and open maximised after a display-scaling issue was observed on Windows.

Verification combines source inspection, earlier automated regression checks, user-confirmed Windows operation and the supplied screenshots. The screenshots directly support bubble-sort correctness, factorial output, stack and queue removal order, linked-list deletion, invalid-input handling and stack underflow. They do not independently prove every possible input or every algorithm. The report therefore distinguishes observed evidence from additional recommended tests.

The recorded times are educational measurements, not controlled performance benchmarks. Sorting and recursion include tracing overhead, whereas stack and queue timings focus on core operations. Complexity tables describe the underlying algorithms; visualisation and snapshot storage add further work and memory.

Contents

Executive Summary . . . . . **i**

List of Acronyms . . . . . **v**

1 Introduction **1**
1.1 Problem Statement 1
1.2 Objectives 1
1.3 Scope 1

2 Requirements and Compliance **2**

3 Development Environment and Architecture **2**
3.1 Technology Choices 3
3.2 Separation of Responsibilities 3
3.3 Execution Flow 4

4 Sorting Algorithms **4**
4.1 Bubble Sort 4
4.2 Selection Sort 4
4.3 Insertion Sort 4
4.4 Merge Sort 5

5 Recursion Algorithms **5**
5.1 Factorial 5
5.2 Naive Fibonacci 5
5.3 Recursive Binary Search 5

6 Custom Data Structures **5**
6.1 Array-Based Stack 6
6.2 Circular Array Queue 6
6.3 Singly Linked List 6

7 Operation Counting and Timing **6**
7.1 Counting Rules 6
7.2 Elapsed-Time Method 7

8 Time and Space Complexity **8**
8.1 Underlying Algorithms 8
8.2 Cost of Recording and Drawing 9

9 Interface Design and Usability **9**

10 Testing and Verification **9**
10.1 Evidence Sources 9
10.2 Screenshot-Backed Results 10
10.3 Recorded Measurement Examples 11

11 Installation and User Guide **11**
11.1 Requirements and Launch 11
11.2 Using the Laboratories 11
11.3 Running the Regression Checks 11
11.4 Version Control and Reproducibility 12

12 Challenges, Limitations and Improvements **12**
12.1 Challenges Addressed 12
12.2 Current Limitations 12
12.3 Possible Future Improvements 12

13 Conclusion **13**

References **14**

A Application Screenshots **15**

List of Figures

1 Overview dashboard and persistent laboratory navigation. 15
2 Bubble-sort input, ascending result and full-run measurements. 16
3 Completed bubble-sort playback at step 12 of 12, with five swaps and ten writes. 17
4 Intermediate bubble-sort step comparing 8 and 5, with active values highlighted. 18
5 Factorial laboratory with input 5 and result 120. 19
6 Factorial metrics and the initial active call at playback step 1 of 10. 20
7 Stack after popping 20: value 10 remains at the top. 21
8 Stack pop steps, measurements and history of push 10, push 20 and pop. . 22
9 Queue after dequeuing 10: value 20 remains. 23
10 Queue physical indices, logical front/rear and dequeue measurements. . . . 24
11 Deletion of the first matching value 20; the remaining list is 30 followed by 10. 25
12 Linked-list deletion explanation showing two visited nodes and one link write. 26
13 Sorting validation rejects the non-integer second item in the input. 27
14 An empty-stack pop reports underflow without removing an item. 28
15 Expanded bubble-sort complexity, counting and timing notes from a separate captured run. 29

List of Acronyms

Term	Meaning
DSA	Data Structures and Algorithms
GUI	Graphical User Interface
JDK	Java Development Kit
JVM	Java Virtual Machine
LIFO	Last In, First Out
FIFO	First In, First Out
IDE	Integrated Development Environment

1. Introduction

Algorithms are procedures for solving problems, while data structures organise the values those procedures manipulate. A correct final answer alone does not explain how a procedure reaches that answer or why its work grows as input size increases. Algorithm Explorer makes these intermediate actions visible.

A user selects a laboratory, enters a small input and runs an algorithm or performs a structure operation. Java classes execute the logic behind the interface. The GUI then displays the answer and explanatory evidence. This is one desktop application: the execution layer runs in the same JVM as the interface, rather than on a separate web server.

1.1. Problem Statement

Students can find it difficult to connect pseudocode, changing data values, recursion calls and asymptotic complexity. Static examples also limit experimentation with duplicates, invalid inputs and empty structures. A unified application is needed to expose these behaviours while preserving the manual implementation of the algorithms and structures being studied.

1.2. Objectives

The main objective is to build an interactive environment for explaining algorithms and data structures through execution.

1. Accept user-defined input and validate it before execution.
2. Implement sorting methods and recursive functions manually.
3. Implement a stack, circular queue and linked list using arrays or custom nodes.
4. Present final results and intermediate execution steps.
5. Count explicitly defined operations and measure elapsed execution time.
6. Explain time and auxiliary-space complexity separately from visualisation overhead.
7. Verify normal behaviour, boundary cases and invalid-input handling.

1.3. Scope

The application is intended for small teaching examples. Sorting and binary-search arrays accept 1–50 integers. Factorial accepts 0–20 so results fit in Java's signed `long`; Fibonacci accepts 0–10 to keep naive recursive playback manageable. The demonstrated stack and queue have capacity five, and the linked list has a display limit of 50 nodes. Session state is retained between laboratories but is not persisted after the application closes. No database, remote server or user-account system is required.

2. Requirements and Compliance

The provided assignment brief requires one graphical or web application covering recursion, data structures, sorting and time complexity. The following mapping relates those requirements to the implementation.

Requirement	**Implementation**
One application	JavaFX workspace containing five laboratories and an overview.
User input	Integer fields, comma-separated arrays and algorithm-selection controls.
Execution behind the interface	Dedicated Java algorithm and structure classes invoked by view handlers.
Results and steps	Final answer cards, recorded sorting/recursion playback, structure explanations and history.
Number of operations	Per-module counters with definitions and exclusions shown in the GUI.
Running time	Differences between `System.nanoTime()` readings, formatted for display.
Recursion	Factorial, naive Fibonacci and recursive binary search.
Data structures	Custom array stack, circular queue and node-based singly linked list.
Sorting	Manually written bubble, selection, insertion and merge sort.
Complexity analysis	Best/worst-case explanations and storage notes in each laboratory.
No built-in sorting or ready-made structures	Source review found no built-in sort calls or ready-made stack, queue or list implementations in the algorithm logic. JavaFX's UI containers remain presentation components.

The review is based on the supplied brief. Any additional marking rubric or submission instructions issued separately by the lecturer must also be checked before submission.

3. Development Environment and Architecture

3.1. Technology Choices

Component	Technology and purpose
Programming language	Java; typed algorithm and structure implementations.
Graphical interface	JavaFX 25; controls, layouts and step playback.
Development environment	Visual Studio Code on Windows 11.
Build tool	Apache Maven; dependencies and compilation.
Java requirement	JDK 25, matching the Maven compiler-release setting.
Version control	Git and GitHub for recording changes and collaboration.

3.2. Separation of Responsibilities

Class/resource	Responsibility
`App.java`	Main window, overview, navigation and cached laboratory views.
`Ui.java`, `theme.css`	Shared presentation helpers, colour scheme, metrics and control styles.
`SortingView.java`	Sorting input, result display, step controls and validation messages.
`SortingAlgorithms.java`	Four sorting implementations and selection of the requested method.
`SortTrace.java`	Step snapshots, comparisons, swaps, writes and elapsed time.
`RecursionView.java`	Recursive-input controls, results, active-call display and playback.
`RecursionAlgorithms.java`	Recursive functions, call tracking and result traces.
`CustomStack.java`	Array-backed stack and operation measurements.
`CustomQueue.java`	Circular-array queue with front, insertion and size tracking.
`CustomLinkedList.java`	Custom nodes, links, insertion, search and first-match deletion.

Class/resource	Responsibility
Structure view classes	Draw current structures and display operation feedback/history.

3.3. Execution Flow

1. The view reads the input and checks its syntax and allowed range.
2. The algorithm/model performs the operation while tracking its defined counters.
3. The execution layer returns a result or trace containing the relevant measurements.
4. The view renders the result and presents explanations.
5. For sorting and recursion, playback moves through already recorded states. Changing playback speed changes the presentation delay, not the recorded algorithm time.

Views remain cached during the session, so switching modules does not discard their state. Playback stops when leaving a laboratory and can be resumed after returning.

4. Sorting Algorithms

4.1. Bubble Sort

Bubble sort compares neighbouring values and swaps an out-of-order pair. Each pass moves a large value toward the end of the unsorted region. A flag stops execution early if a complete pass performs no swaps. This produces a linear best case on already sorted input, while average and worst cases are quadratic.

For the captured input $[8, 3, 5, 1, 9]$, the final result is $[1, 3, 5, 8, 9]$. The screenshot records 10 comparisons and five swaps. Each swap writes two array positions, producing 10 writes and 20 counted operations. A playback snapshot shows a comparison between 8 and 5 and the resulting intermediate array $[3, 5, 8, 1, 9]$.

4.2. Selection Sort

Selection sort scans the remaining unsorted region for its minimum and places that value at the next output position. It repeats the scan even when the input is already sorted. The number of value comparisons is therefore quadratic in best, average and worst cases. Only necessary exchanges contribute swap writes.

4.3. Insertion Sort

Insertion sort holds a key, shifts larger elements to the right and inserts the key into the correct position in the sorted prefix. The display can show the held key because it

temporarily exists outside the changing array positions. Already sorted input requires linear work; reverse-sorted input requires a quadratic number of comparisons and movements.

4.4. Merge Sort

Merge sort recursively divides the array into smaller ranges, sorts the ranges and merges them using a temporary array. There are logarithmically many levels of division, each involving linear total merge work. The implementation allocates the merge buffer before timing begins. Temporary-buffer writes are included in its write counter. The GUI primarily displays snapshots of the main array and explains temporary-buffer activity in the step text.

5. Recursion Algorithms

5.1. Factorial

Factorial uses the recurrence

$$F(n) = \begin{cases} 1, & n \leq 1, \\ nF(n-1), & n > 1. \end{cases}$$

For input 5, the function calls factorial for 5, 4, 3, 2 and 1, then returns values 1, 2, 6, 24 and 120. The supplied screenshot confirms $5! = 120$, five calls, maximum active depth five and 14 counted operations. These 14 operations comprise five calls, five base-case checks and four multiplications. The active-call panel in the screenshot is at step 1 of 10; it shows the initial call rather than the deepest stack state.

5.2. Naive Fibonacci

The implementation uses $F(0) = 0$, $F(1) = 1$ and $F(n) = F(n-1) + F(n-2)$ for larger inputs. This deliberately exposes repeated subproblems: it is not memoised. Its time has an exponential upper bound $O(2^n)$, while the maximum live recursion depth is $O(n)$. The small input limit prevents the trace from becoming impractically large.

5.3. Recursive Binary Search

Binary search requires an ascending array. It compares the target with the middle value, returns a matching index or recursively searches one half. If the search interval becomes empty, it reports that the target is absent. Validation rejects unsorted input instead of silently sorting it. Validation and input copying take $O(n)$ before the timed search; the search itself takes $O(\log n)$ in the worst case.

6. Custom Data Structures

6.1. Array-Based Stack

The stack stores integers in a fixed-size array and tracks a top index. An empty stack has top index -1. Push checks capacity, advances the index and writes a value. Pop checks emptiness, reads the top item and decreases the index. Peek reads the top without removing it.

The evidence shows push 10, push 20 and pop. The removed value is 20 and the remaining top is 10, demonstrating LIFO behaviour. A successful pop counts one boundary check, one read and one index update: three operations. Popping an empty stack returns an underflow message and leaves the structure unchanged.

6.2. Circular Array Queue

The queue uses an array, a front index, a next-insertion index and a stored size. Advancing an index uses modulo capacity, allowing reuse of positions at the beginning of the array. Size distinguishes full and empty states when index positions coincide. Dequeue does not shift the remaining values.

The captured sequence enqueues 10 and 20, then dequeues 10. Value 20 remains at physical index 1 and is both the logical front and rear. Next insertion is at index 2. The dequeue counts one check, one read, one front-index update and one size update: four operations. This demonstrates FIFO behaviour.

6.3. Singly Linked List

Each custom node contains an integer value and a reference to the next node. A head reference identifies the first node; the last node points to null. Insert First changes the head links. Insert Last traverses to the final node because this implementation has no stored tail reference. Search visits nodes sequentially, and Delete First Match reconnects links around the first matching node.

The supplied deletion evidence shows a visit to value 30 at index 0 and value 20 at index 1, followed by bypassing the matching node. The final list is $30 \rightarrow 10 \rightarrow$ null. Two visits and one structural link write give three counted operations. This captured order differs from the earlier suggested Insert Last demonstration; the report records the observed order and does not infer the exact insertion-button sequence.

7. Operation Counting and Timing

7.1. Counting Rules

Counts are deliberately selected educational operations, not a count of every JVM or processor instruction. A zero value in one category does not imply that no computation occurred.

Module	Counted operations	Main exclusions
Sorting	Value comparisons plus array writes; a swap adds two writes.	Loop checks, index changes, local assignments and trace copies.
Recursion	Calls, base-case checks, multiplications, additions and value comparisons.	Argument/index arithmetic, local assignments and trace operations.
Stack	Boundary checks, array reads/writes and top-index updates.	Local assignments, return handling and drawing.
Queue	Boundary checks, array reads/writes, stored-index updates and size updates.	Arithmetic, local assignments and drawing.
Linked list	Nodes visited plus structural writes to head/next links.	Local pointer movement, size updates, boundary checks and default node initialisation.

7.2. Elapsed-Time Method

The program subtracts a start reading from a finish reading of `System.nanoTime()`. The result is a duration, not a wall-clock timestamp. Nanosecond units do not guarantee nanosecond measurement accuracy.

```
long start = System.nanoTime();
// Execute the measured operation.
long elapsedNanoseconds = System.nanoTime() - start;
```

Sorting includes counters and step recording, but excludes initial setup, the initial/final snapshots and playback. Recursion includes tracing but excludes validation and initial allocation/copying. Stack and queue timing excludes explanation creation. Linked-list timing includes explanation recording and excludes the later display snapshot and rendering.

These scopes differ, so raw times across laboratories are not directly comparable. JVM warm-up, garbage collection, operating-system scheduling and machine load also affect results. For example, the bubble-sort result screenshot displays 3.85 ms, whereas the separate complexity screenshot records 46,900 ns. They are separate observations and must not be combined as a single run. Neither observation alone establishes a complexity class.

8. Time and Space Complexity

8.1. Underlying Algorithms

The following bounds describe the core algorithms, excluding GUI and recorded-trace storage. Here n is input length for sorting/search and the input integer for factorial/Fibonacci; c is fixed structure capacity.

Algorithm / operation	**Best time**	**Worst time**	**Auxiliary space**
Optimised bubble sort	$O(n)$	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$ buffer; $O(\log n)$ stack
Factorial	$O(n)$	$O(n)$	$O(n)$ call stack
Naive Fibonacci	$O(2^n)$ upper bound; base cases $O(1)$		$O(n)$ call stack
Recursive binary search	$O(1)$	$O(\log n)$	$O(\log n)$ stack
Stack push/pop/peek	$O(1)$	$O(1)$	$O(1)$ per operation
Queue enqueue/dequeue/front/rear	$O(1)$	$O(1)$	$O(1)$ per operation
List Insert First	$O(1)$	$O(1)$	$O(1)$ per inserted node
List Insert Last	$O(1)$ empty	$O(n)$	$O(1)$ per inserted node
List Search/Delete	$O(1)$	$O(n)$	$O(1)$ excluding explanation

Average-case sorting time is $O(n^2)$ for bubble, selection and insertion sort and $O(n \log n)$ for merge sort under the usual input assumptions. Total stack/queue storage is $O(c)$; total linked-node storage is $O(n)$. Bounds for fixed-capacity teaching displays describe the generalised algorithms as their capacity/input scale.

8.2. Cost of Recording and Drawing

Saving an entire length-n array for each of $O(n^2)$ sorting events can require $O(n^3)$ stored values in the worst case for the quadratic sorting methods. Merge-sort snapshots can require $O(n^2 \log n)$ stored values. The trace also reserves an $O(n^2)$ array of step references. Thus the complete teaching application does not have the same space usage as a plain in-place sort.

Factorial call-stack snapshots can require $O(n^2)$ trace storage. Fibonacci tracing has an $O(n2^n)$ upper bound, and binary-search stack snapshots can contain $O((\log n)^2)$ entries. String formatting adds representation costs. Drawing the stack/list is linear in displayed items; drawing the queue's physical array is linear in capacity. The application limits input size to keep this additional work manageable.

9. Interface Design and Usability

The navy sidebar provides a stable location for navigation, while teal buttons identify primary actions. White cards group controls, results, metrics and visualisation. Active values use amber highlighting; completed sorting uses green. Text labels accompany colours to explain their meaning.

Playback includes Play, Pause, Previous, Next, Restart Steps and Last Step, with a speed control. Invalid input produces a message beside the input controls. Structure resets request confirmation when values or history would be cleared. Explanatory notes are collapsible so the main operation stays visible.

The original fixed-size window extended beyond the visible desktop under the user's display setup. The corrected startup uses JavaFX visual screen bounds, caps the restored window to 94% of the available width/height (with upper limits of 1360 by 900 logical units), centres it and opens maximised. The user subsequently confirmed the fix on Windows. These settings address window placement; they do not imply that every screen resolution has been tested.

10. Testing and Verification

10.1. Evidence Sources

- Source inspection checked the current algorithm and structure implementation and its measurement definitions.
- An earlier regression-runner execution reported 42,990 successful assertions, including generated sorting inputs, counters, boundary cases, queue wraparound, list deletion and recursive examples. These assertions are checks within a runner, not 42,990 independent end-to-end test cases.

- Earlier JavaFX smoke checks covered navigation, retained state, playback, validation and laboratory screens in a Linux display environment.
- The user confirmed that all laboratories and the startup-sizing fix worked on Windows. The screenshots below provide direct visual evidence for the specific captured states.

10.2. Screenshot-Backed Results

ID	Test	Observed result	Status
T01	Overview/navigation	Dashboard and all five laboratory links are visible.	Observed
T02	Bubble sort: 8, 3, 5, 1, 9	Sorted output 1, 3, 5, 8, 9; 10 comparisons, 10 writes, 20 counted operations.	Pass
T03	Sorting playback	Step 3/12 shows comparison/swap of 8 and 5; final screenshot shows step 12/12.	Pass
T04	Factorial: 5	Result 120; five calls, depth five and 14 counted operations.	Pass
T05	Stack: push 10, push 20, pop	20 removed; 10 remains; three counted operations.	Pass
T06	Queue: enqueue 10, enqueue 20, dequeue	10 removed; 20 remains at index 1; four counted operations.	Pass
T07	Linked list: delete first 20	Matched node at index 1 removed; final order 30 then 10; three counted operations.	Pass
T08	Sorting: 8, hello, 3	Item 2 rejected as a non-integer; no result shown.	Pass
T09	Pop from empty stack	Underflow message; size remains zero and top index is -1.	Pass
T10	Complexity explanation	Bubble-sort bounds, counting exclusions and timing caveats visible.	Observed

The screenshot set contains bubble sort and factorial examples rather than visual proof for every sorting and recursive method. Additional useful manual checks are duplicates/negative numbers, all other sorting choices, factorial zero, Fibonacci 10, successful/unsuccessful binary search, unsorted-search input, structure overflow and queue wraparound. Their

expected outcomes should not be recorded as screenshot-observed results without further evidence.

10.3. Recorded Measurement Examples

Captured operation	Count	Displayed elapsed time
Bubble-sort result	20	3.85 ms
Factorial of 5	14	28.96 ms
Stack pop	3	3.30 microseconds
Queue dequeue	4	4.50 microseconds
Linked-list deletion	3	509.10 microseconds

These values are transcribed from the supplied captures. They are examples of instrumented runs, not averages or evidence that one laboratory is faster than another.

11. Installation and User Guide

11.1. Requirements and Launch

Install JDK 25 and Apache Maven, and ensure their commands are available in the terminal. Open the project folder containing `pom.xml` in VS Code. Check the environment and run:

```
java --version
javac -version
mvn -version
mvn clean javafx:run
```

An Internet connection is required when Maven first downloads dependencies. After successful setup, algorithm execution itself does not require a network connection. The source archive is not a standalone installer or self-contained executable.

11.2. Using the Laboratories

Select a laboratory from the sidebar. For sorting, choose a method, enter comma-separated integers and click Run. Inspect the final result, then use playback to follow steps. For recursion, choose a function and enter its integer input; binary search also needs an ascending array and a target. For structures, enter an integer and choose an insertion/removal/query operation. Expand the measurement notes to understand what the counters include. Scroll when cards extend below the visible area.

11.3. Running the Regression Checks

The supplied regression runner is a plain Java main class, not a JUnit suite. On Windows:

```
mvn test-compile
```

```
java -cp "target/classes;target/test-classes" com.algorithmexplorer.
    AlgorithmRegressionChecks
```

On Linux/macOS, use a colon instead of a semicolon between classpath entries. Running `mvn test` alone does not execute this plain-main regression runner.

11.4. Version Control and Reproducibility

The project repository is https://github.com/eram-ok/algorithm-explorer. Git records source changes and provides a shared history. Before submission, build a fresh copy using the README, retain the source and `pom.xml`, and exclude generated build output such as `target/` from version control. Access permissions must allow the intended reviewer to obtain the repository or a supplied source archive.

12. Challenges, Limitations and Improvements

12.1. Challenges Addressed

The initial setup required installing Maven and making it available on the Windows command path. The JavaFX prototype was expanded into a shared interface with consistent controls and result cards. Navigation was updated to retain laboratory state, and a screen-sizing problem was resolved by using available screen bounds and maximised startup. These changes were confirmed through the user's setup and execution feedback.

12.2. Current Limitations

- Trace recording increases time and memory beyond the underlying algorithms.
- Single-run timings are sensitive to JVM and system effects and use different measurement scopes across modules.
- Input and capacity limits intentionally restrict large demonstrations.
- Session data is lost when the program closes; there is no save/load facility.
- Long explanations and visualisations require scrolling, particularly under display scaling.
- The provided screenshots do not document all possible boundary cases or every algorithm choice.

12.3. Possible Future Improvements

Future versions could add a separate benchmark mode with tracing disabled, repeated runs and clearly documented warm-up. Other possible improvements include exporting traces, saving sessions, automated GUI checks, additional sorting methods and more accessible keyboard-driven visualisation. These are proposed extensions, not features claimed for the submitted version.

13. Conclusion

Algorithm Explorer brings sorting, recursion and custom data structures into one JavaFX application. Users can provide their own inputs, inspect results and follow the operations that produced them. The implementation also explains selected operation counts, measured elapsed time and theoretical complexity.

The reviewed source and supplied evidence support the core requirements of the provided brief. The most important interpretation is that algorithm complexity and instrumented application performance are different: snapshots, explanations and drawing add work. By documenting this distinction and recording actual observed outcomes, the project provides an understandable and reproducible learning tool.

References

[1] CSC2107. *End of Semester Group Project: Algorithm Explorer*. Course assignment brief supplied to the group, 2026.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. Fourth edition. MIT Press, 2022. Course reference for sorting, recursion, data structures and asymptotic analysis.

[3] Algorithm Explorer project. Java source files, Maven configuration and regression runner, November 2026. `https://github.com/eram-ok/algorithm-explorer`.

[4] Project group. *Algorithm Explorer screenshot evidence*. Fifteen PNG files supplied for this report, November 2026.

A. Application Screenshots

The following figures retain the supplied image filenames and show the actual captured application states. They are not reconstructed interface illustrations.

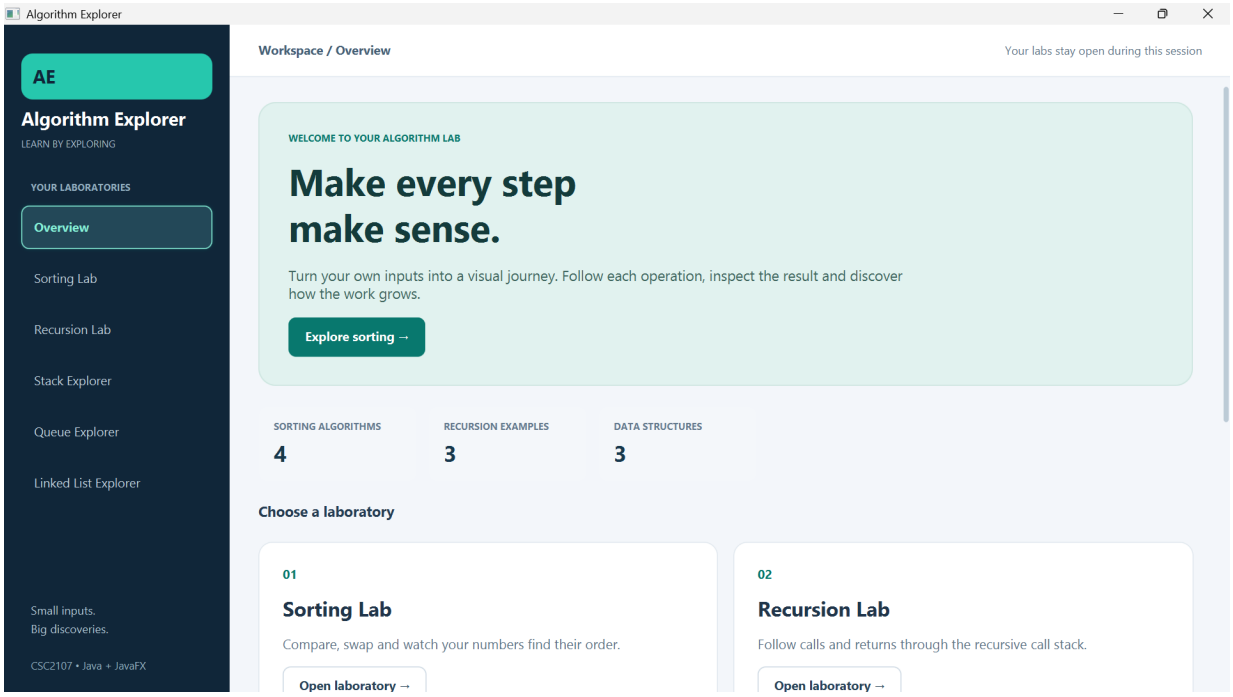


Figure 1: Overview dashboard and persistent laboratory navigation.

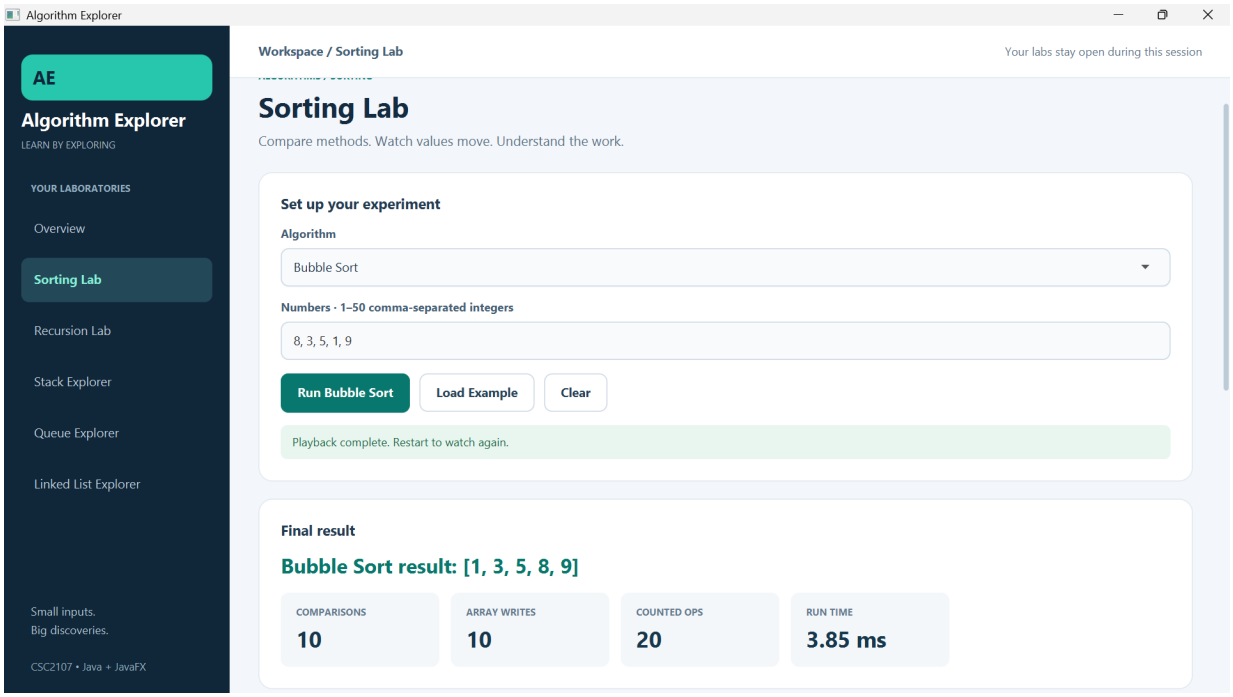


Figure 2: Bubble-sort input, ascending result and full-run measurements.

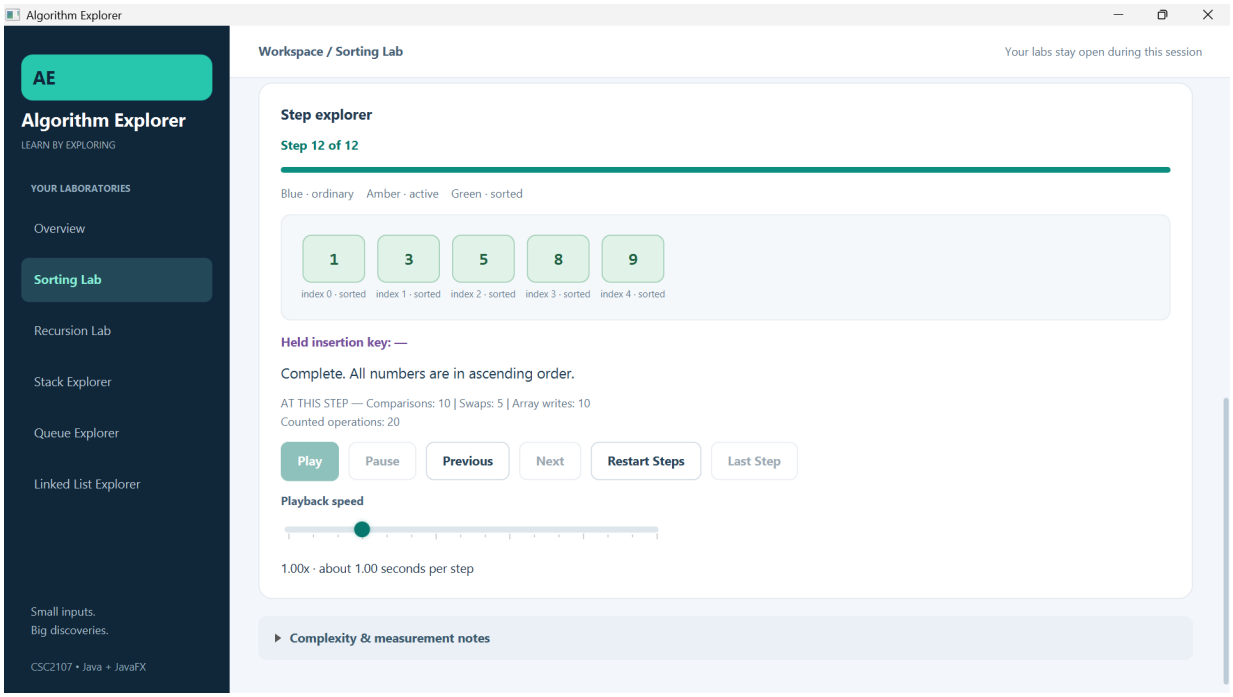


Figure 3: Completed bubble-sort playback at step 12 of 12, with five swaps and ten writes.

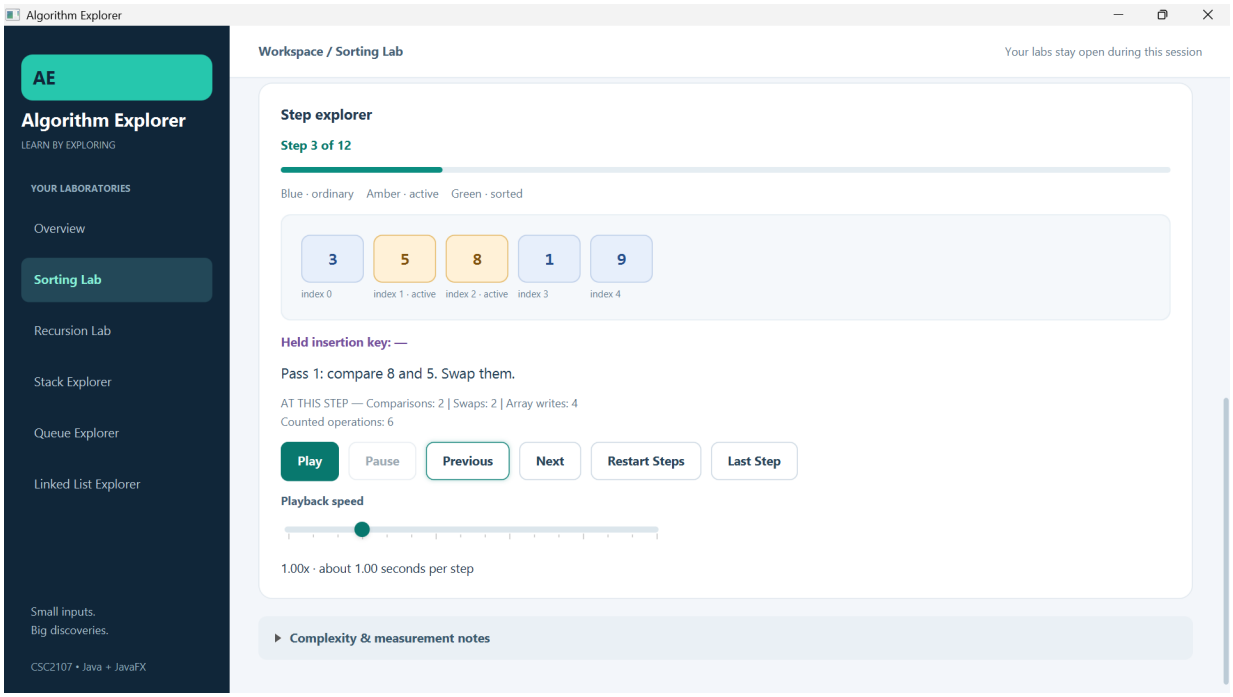


Figure 4: Intermediate bubble-sort step comparing 8 and 5, with active values highlighted.

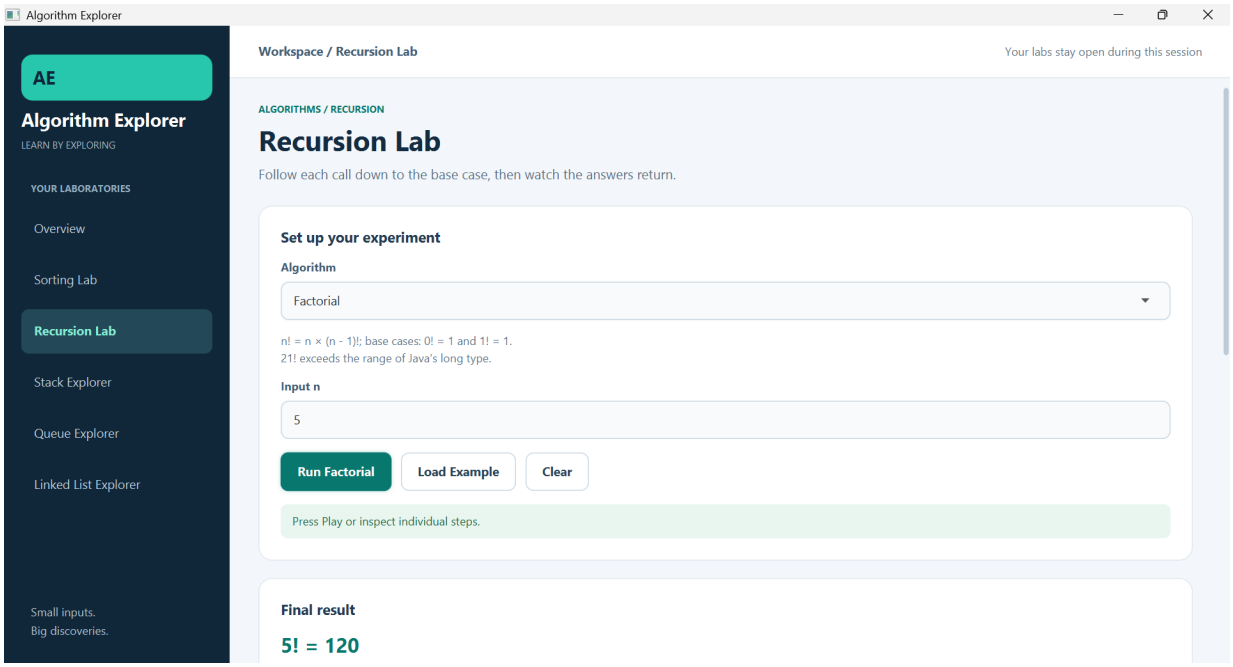


Figure 5: Factorial laboratory with input 5 and result 120.

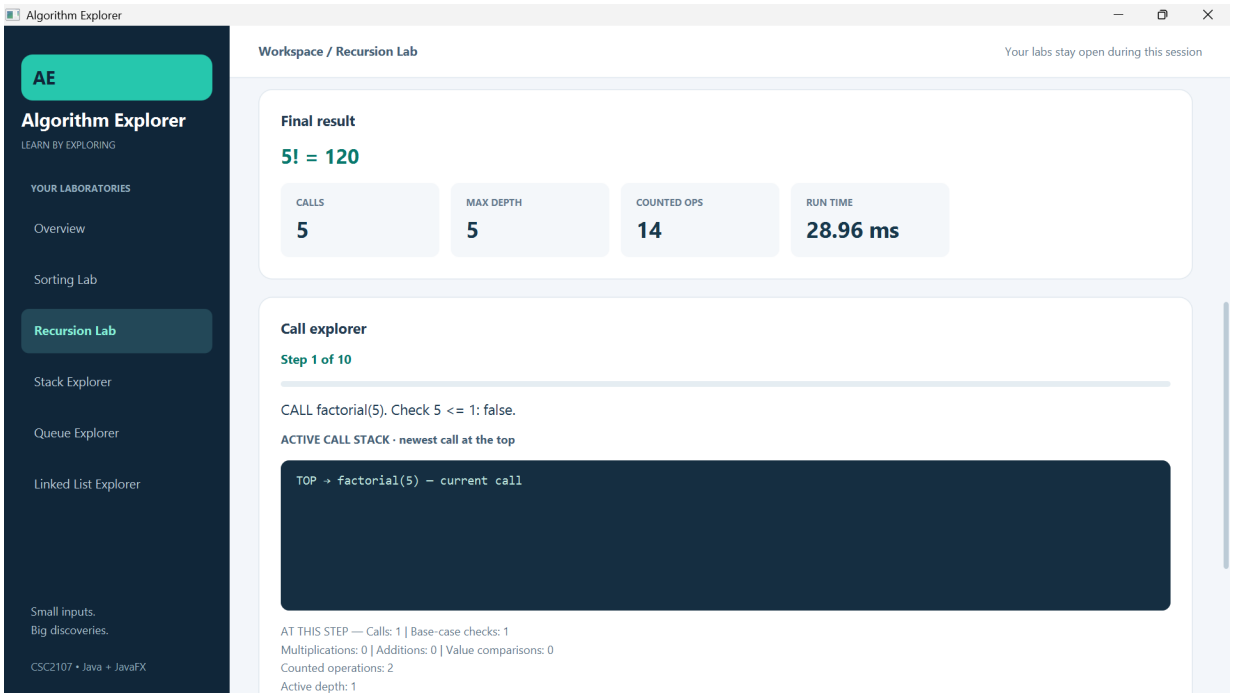


Figure 6: Factorial metrics and the initial active call at playback step 1 of 10.

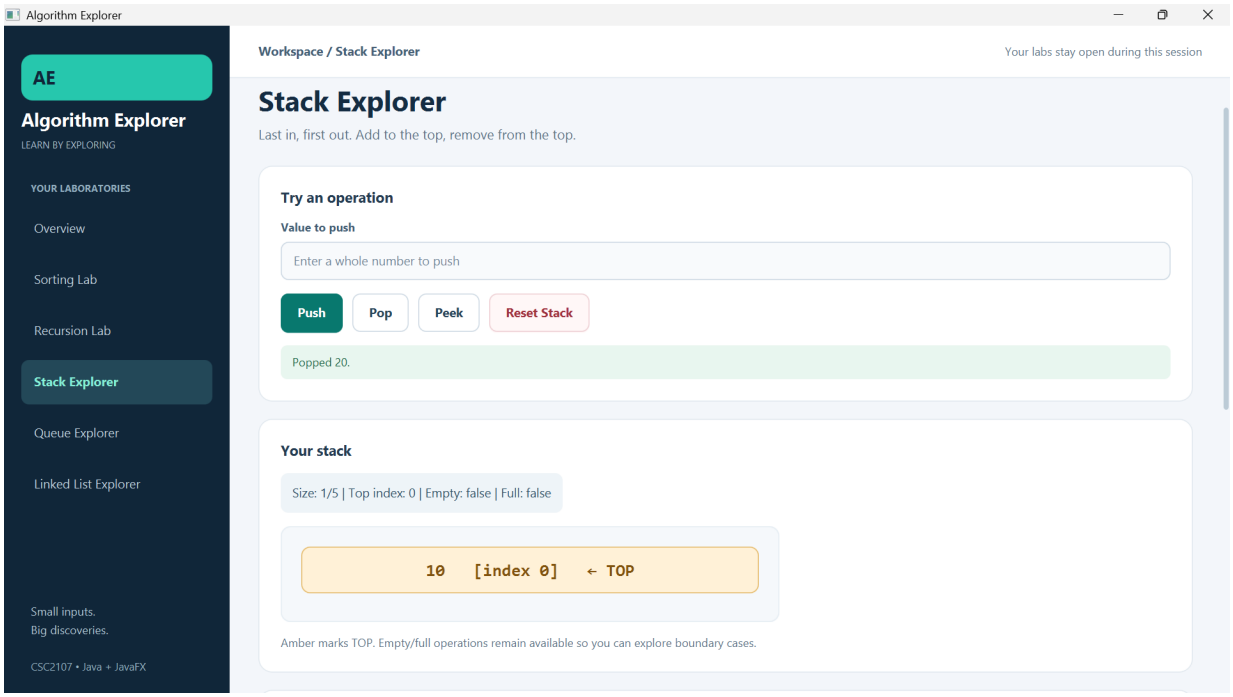


Figure 7: Stack after popping 20: value 10 remains at the top.

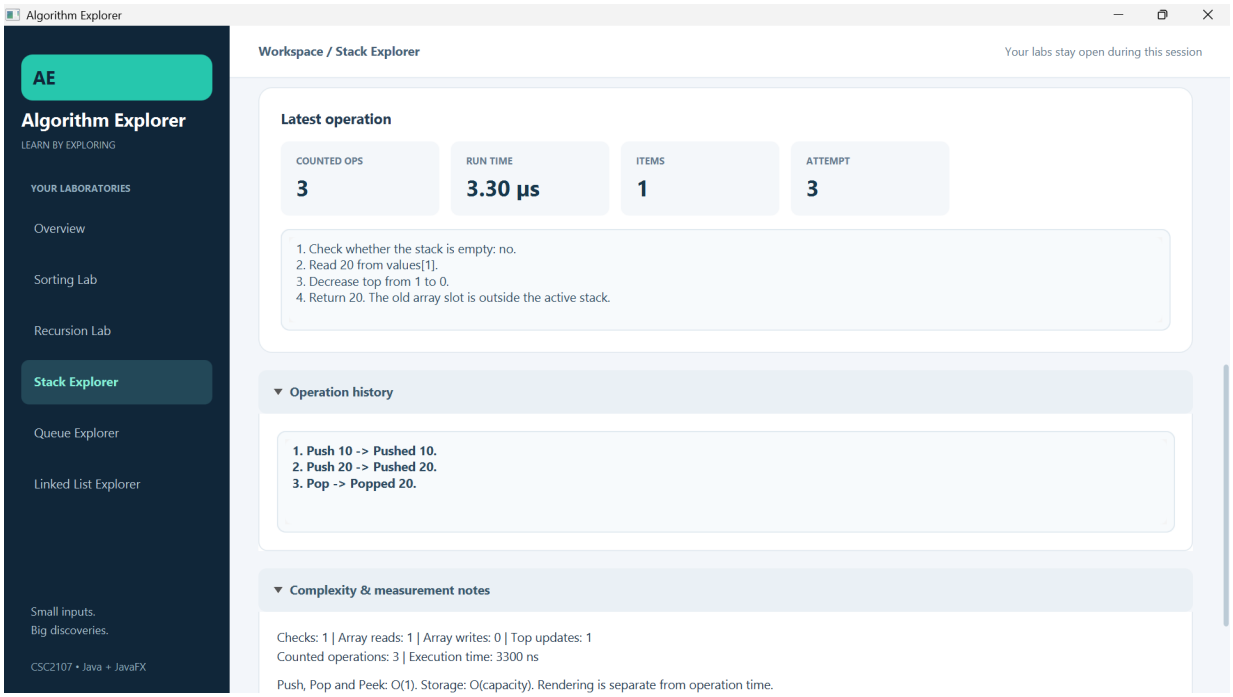


Figure 8: Stack pop steps, measurements and history of push 10, push 20 and pop.

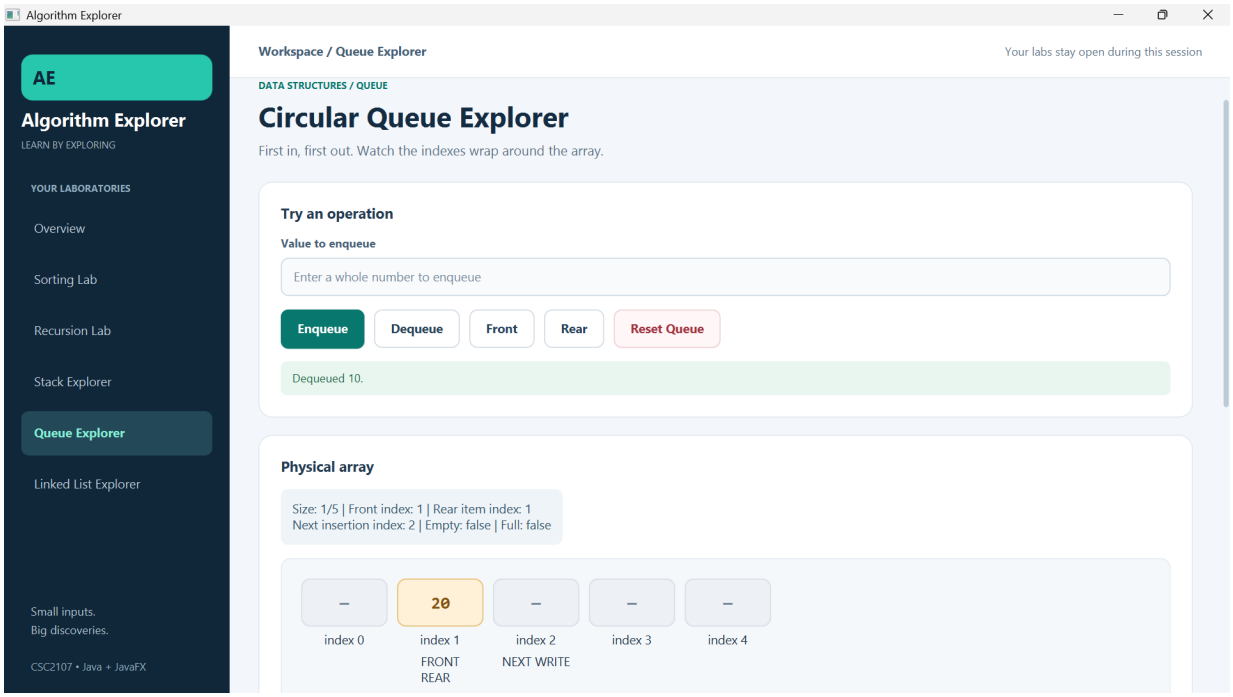


Figure 9: Queue after dequeuing 10: value 20 remains.

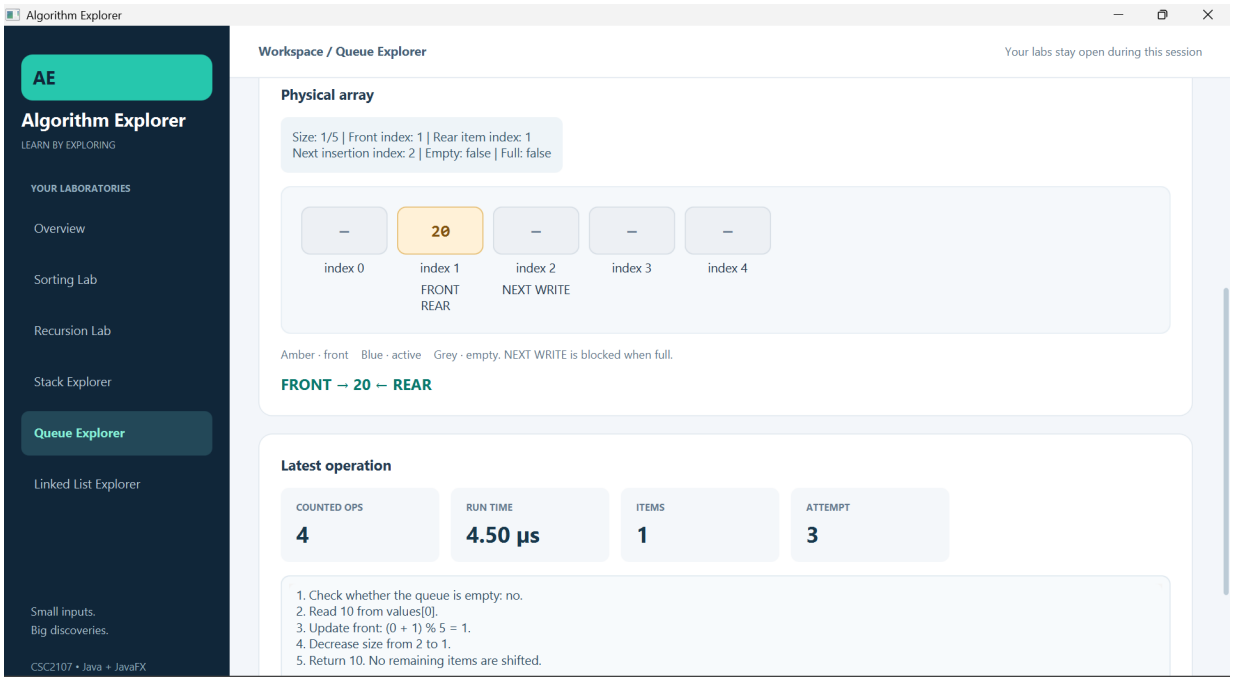


Figure 10: Queue physical indices, logical front/rear and dequeue measurements.

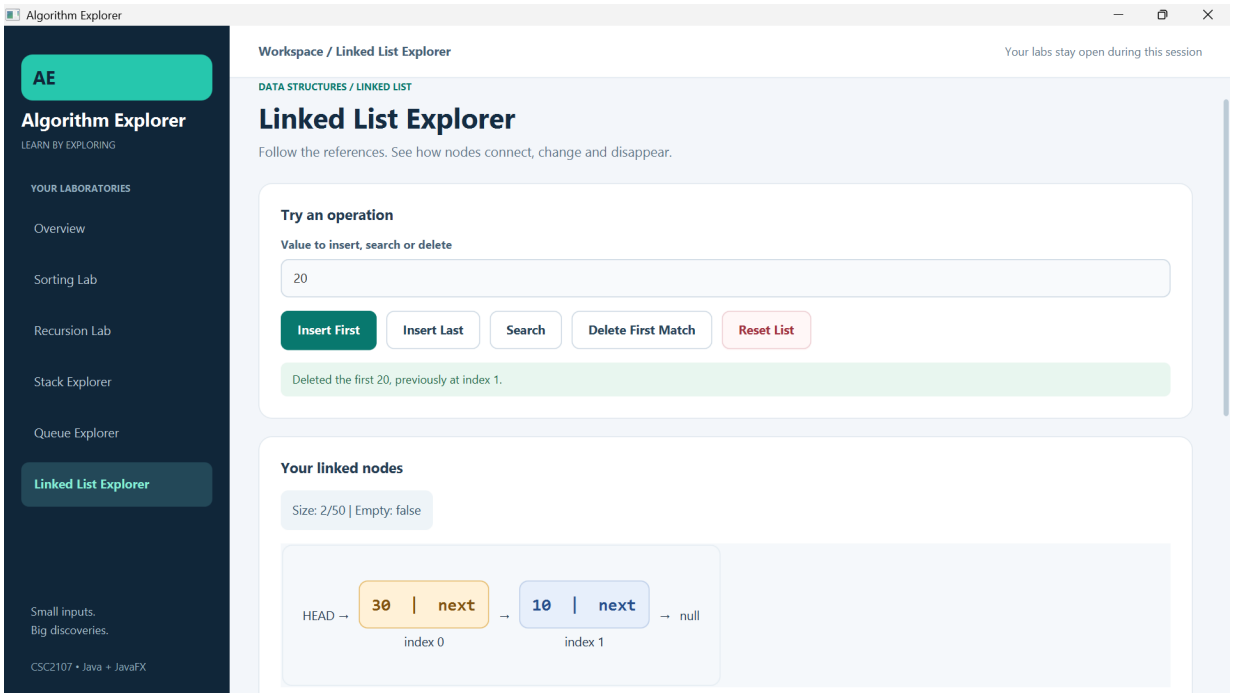


Figure 11: Deletion of the first matching value 20; the remaining list is 30 followed by 10.

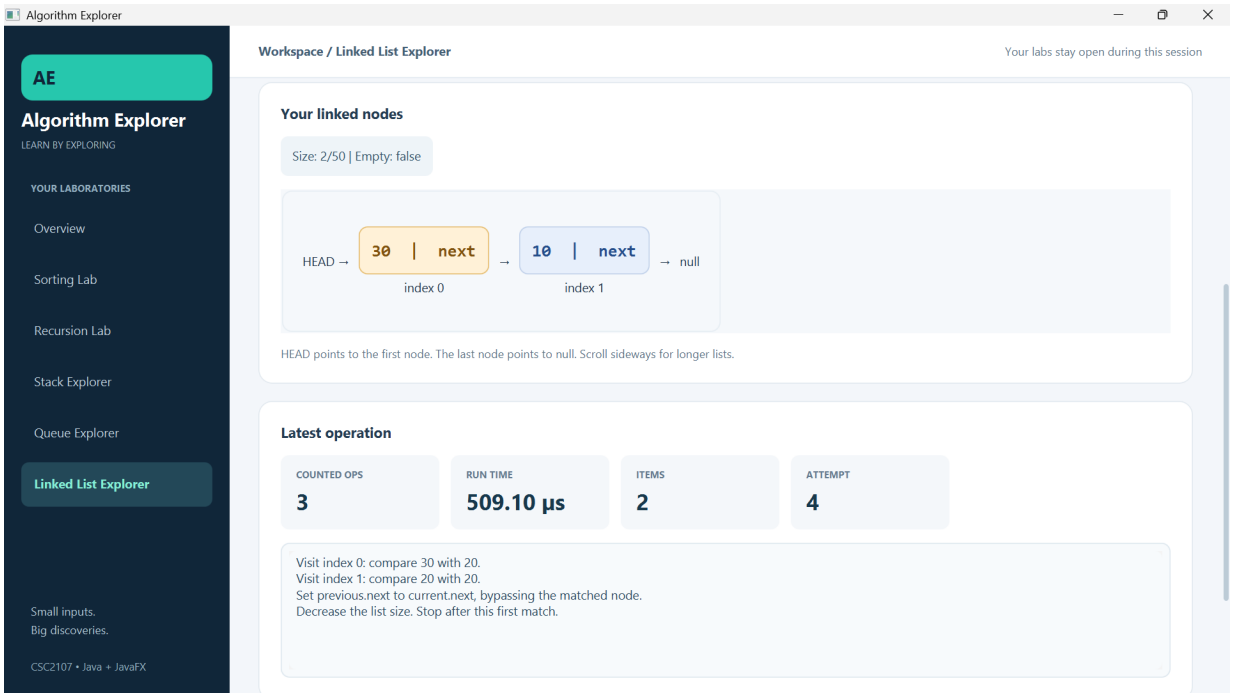


Figure 12: Linked-list deletion explanation showing two visited nodes and one link write.

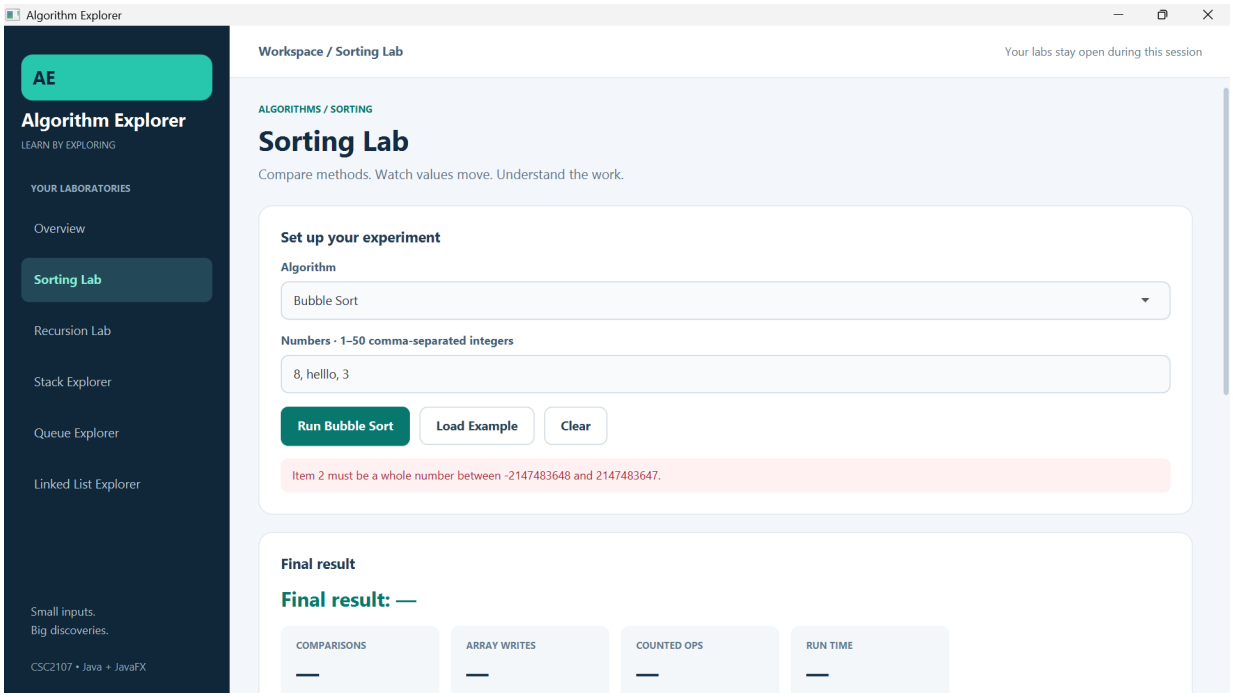


Figure 13: Sorting validation rejects the non-integer second item in the input.

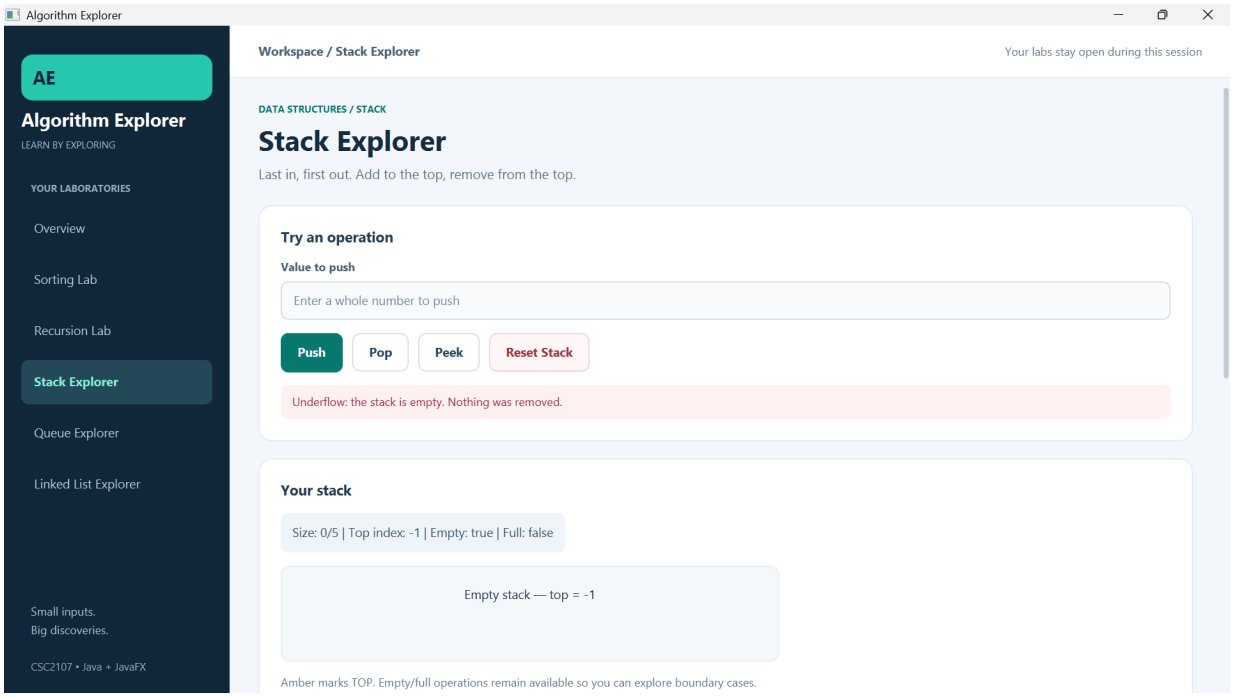


Figure 14: An empty-stack pop reports underflow without removing an item.

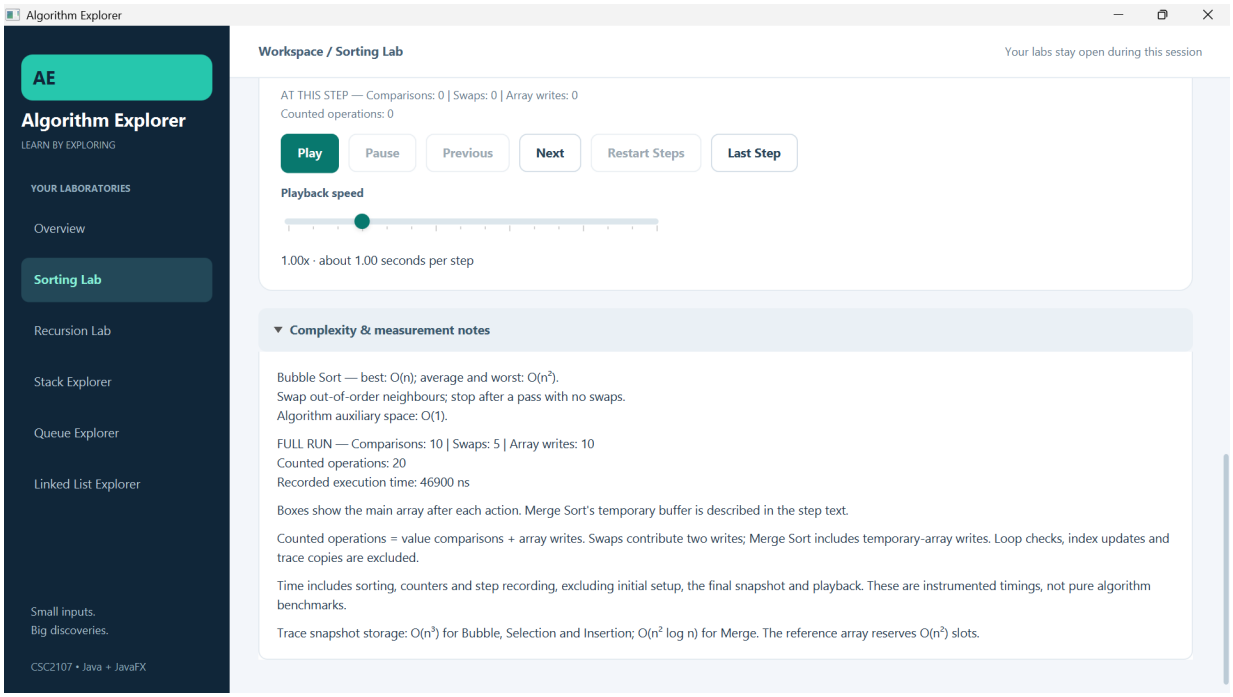


Figure 15: Expanded bubble-sort complexity, counting and timing notes from a separate captured run.